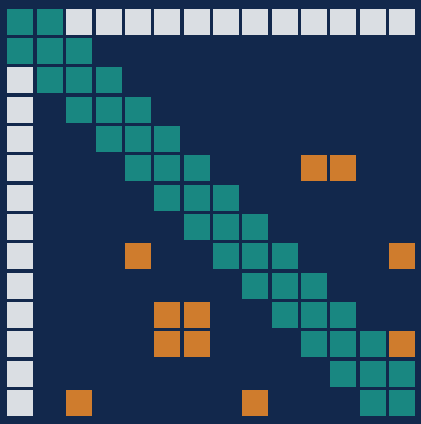
TECHNICAL REVIEW | EFFICIENT TRANSFORMERS



BigBird

Sparse-Attention Transformers for Long Sequences

Architecture, motivation, key innovations, complexity comparison with BERT and Longformer, applications and limitations

Primary source: Zaheer et al., "Big Bird: Transformers for Longer Sequences", Advances in Neural Information Processing Systems 33 (NeurIPS 2020), arXiv:2007.14062

Prepared October 2026

Contents

1. Executive Summary 2

2. Background and Motivation 2

3. BigBird Architecture 3

4. Block-Sparse Implementation 4

5. Theoretical Guarantees 5

6. Computational Complexity: BERT vs Longformer vs BigBird 6

7. Performance Characteristics 7

8. Suitable Applications for Long Documents 8

9. Advantages and Limitations 9

10. Conclusion 10

References 10

About this document

This review explains how BigBird reduces the quadratic cost of Transformer self-attention, and how that compares with BERT and Longformer. Architecture details, hyperparameters and benchmark numbers come from the BigBird paper (Zaheer et al., 2020) and its appendix. Figures marked *illustrative* or *derived* are my own arithmetic or drawings based on those published settings, not measurements. Longformer's properties are taken from its original paper (Beltagy et al., 2020).

1. Executive Summary

Standard Transformers such as BERT let every token attend to every other token. That is powerful but makes compute and memory grow with the **square** of the sequence length, which in practice limits common models to roughly 512 tokens. Many real tasks, such as question answering over several documents, summarizing papers or patents, and modelling DNA, need far more context.

BigBird, introduced by Google Research in 2020, replaces full attention with a sparse pattern that has three ingredients: a handful of **global tokens** that see everything, a **sliding window** of local neighbours, and a few **random** connections. Because each token attends to a fixed number of keys, cost grows **linearly** with length. The authors report handling sequences up to 8x longer than previously possible on similar hardware, and prove that the sparse model keeps the theoretical expressiveness of full attention.

Key facts at a glance

- **Problem:** full self-attention is $O(n^2)$ in time and memory.
- **Solution:** sparse attention = global + window + random, O(n) for fixed g, w, r.
- **Hardware trick:** attention is computed on *blocks* of tokens so GPUs/TPUs can use dense matrix products.
- **Theory:** universal approximator of sequence functions and Turing complete; global tokens are essential to the proof.
- **Results:** state of the art on several QA datasets, large gains in long-document summarization, and new results on genomics at the time of publication.
- **Caveat:** no free lunch. Some tasks provably need many more layers under any sparse pattern, and for short inputs full attention is the better choice.

2. Background and Motivation

2.1 Why full self-attention is expensive

In self-attention each of the n tokens produces a query that is compared with the keys of all n tokens, producing an n x n score matrix. The BigBird paper views this as a **complete directed graph** in which every token is connected to every other token; BERT corresponds to an adjacency matrix of all ones. Both the number of inner products and the memory to hold the scores scale as n^2, and the authors note that on commonly available hardware this limits inputs to roughly 512 tokens.

The practical consequence is that tasks needing wide context must either truncate the input or split it into pieces. Chunking and retrieve-then-read pipelines work, but the paper points out they typically require significant engineering and can be hard to train, and information that straddles chunks is lost.

2.2 Two questions BigBird sets out to answer

- Can we get the empirical benefits of full attention using far fewer inner products?
- Do such sparse mechanisms preserve the expressive power and flexibility of the original Transformer?

Earlier efficient-attention work was largely heuristic. The authors observe that most such methods lack theoretical guarantees and often do not match full Transformers across many benchmarks with a single architecture. Longformer is called out as the main exception that is empirically competitive, and it is included in all comparisons.

2.3 Where BigBird sits among related approaches

Approach	Idea	Cost
Sparse Transformer (Child et al., 2019)	Fixed strided/local sparse patterns	O(n sqrt(n))
Reformer (Kitaev et al., 2020)	Locality-sensitive hashing to find nearest keys	O(n log n)
Longformer (Beltagy et al., 2020)	Sliding window plus a few global tokens	O(n w)
ETC (Ainslie et al., 2020)	Global tokens to encode structure; direct precursor of BigBird-ETC	Linear
BigBird (Zaheer et al., 2020)	**Global + window + random, with theory**	**O(n)**

Costs for Sparse Transformer and Reformer are as quoted in the BigBird paper's related-work section.

3. BigBird Architecture

3.1 Attention as a sparse graph

BigBird keeps the standard Transformer encoder layer (multi-head attention followed by a feed-forward network, with residual connections). Only the attention pattern changes. For each token i, a neighbour set N(i) defines which keys it may attend to, and the softmax is taken over those keys only. Choosing N(i) for all tokens defines a directed graph; if it is the complete graph you recover ordinary attention.

The design is guided by graph sparsification. A good sparse attention graph should have (1) a **short average path length** between any two tokens, so information can travel quickly, and (2) **locality**, because in language and genomics much of what matters about a token is in its neighbours. Random graphs give the first property; sliding windows give the second. The paper draws on Erdos-Renyi random graphs (shortest paths are logarithmic in the number of nodes) and Watts-Strogatz small-world graphs (high clustering plus short paths).

3.2 The three building blocks

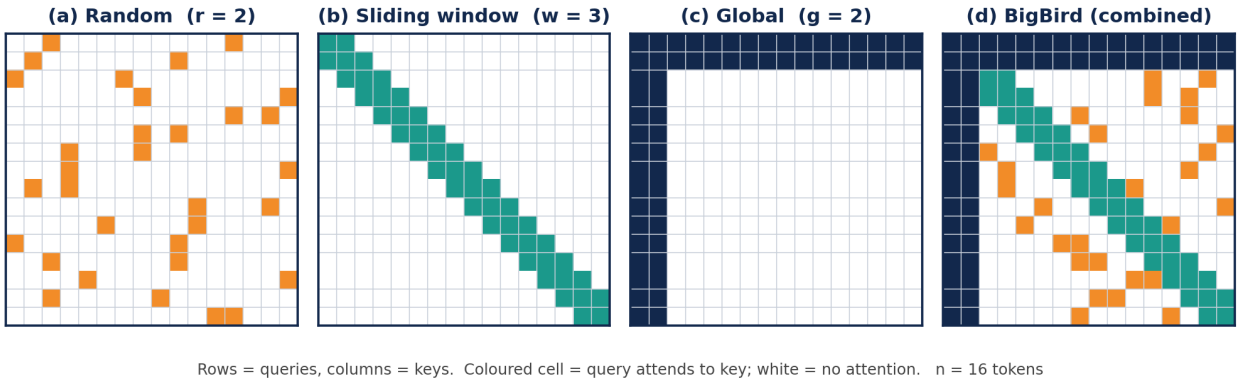


Figure 1. The attention building blocks of BigBird at token level, redrawn for illustration (random cells are arbitrary samples).

Component	What each query attends to	Role
Random attention	r randomly chosen keys	Creates shortcuts across the sequence; keeps average path length short (expander-like behaviour)
Sliding window	w/2 neighbours on each side (w total)	Captures local context; exploits locality of reference
Global tokens	g tokens attend to *all* tokens and all tokens attend to them	Act as information hubs (like [CLS]); critical for the theory and for performance

A sanity check in the paper shows why all three are needed. At 512 tokens with a BERT-base setup, random-only and window-only attention fall clearly behind full attention, and combining random and window recovers part of the gap, but global tokens are what close the remaining distance.

Model (length 512)	MLM	SQuAD	MNLI
BERT-base (full attention)	64.2	88.5	83.4
Random (R)	60.1	83.0	80.2
Window (W)	58.3	76.4	73.1
R + W	62.7	85.1	80.5

Table 1. Building-block ablation (Zaheer et al., 2020, Table 1). The paper states that random blocks and a local window alone were insufficient to match BERT.

3.3 Global tokens: ITC versus ETC

The paper defines two ways of providing global tokens:

- **BigBird-ITC (Internal Transformer Construction):** a subset of the *existing* tokens is made global. No tokens are added to the sequence.
- **BigBird-ETC (Extended Transformer Construction):** *extra* global tokens (such as [CLS]) are added to the sequence, giving the model additional places to store context. In the paper's experiments this variant performed best.

For question answering, the ETC configuration assigns global tokens by structure, for example one per question token, one per evidence paragraph and one per sentence in HotpotQA. This means ETC needs some task-specific design, whereas ITC works on any text.

4. Block-Sparse Implementation

4.1 Why "blockify"?

A sparse pattern on paper does not automatically run fast. The authors note that GPUs and TPUs perform best on coalesced memory access and dense matrix products, and that sporadic lookups from a sliding window or random keys are inefficient. Their solution is to pack queries and keys into **blocks** of b consecutive tokens and define random, window and global attention over blocks instead of individual tokens.

4.2 How the computation works

- **Window:** reshape queries and keys into (n/b) x b x d tensors. Make w copies of the key tensor and "roll" each by one block, so one batched matrix product yields all window scores without a gather.
- **Global:** the first g key blocks are simply concatenated, giving a fixed set of columns.
- **Random:** the only part that needs a (less efficient) gather operation; it is small (r = 3 in all the paper's experiments).

The three parts are concatenated into one compact key tensor of size (n/b) x (g+w+r)b x d. Multiplying it with the blocked query tensor costs **O(n (g+w+r) b d)**, versus $O(n^2 d)$ for dense attention, and is a plain dense multiplication that accelerators handle well.

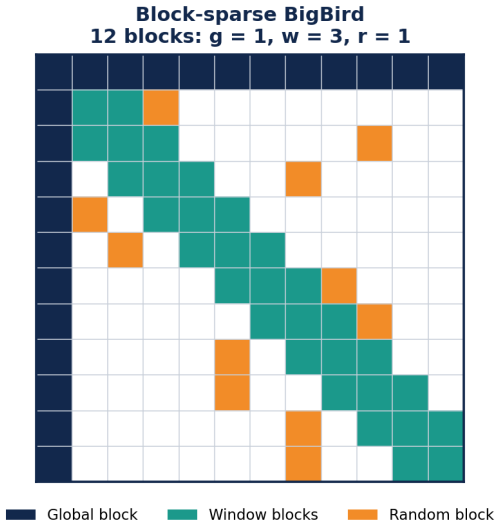


Figure 2. Block-level attention matrix (12 blocks, g = 1, w = 3, r = 1), redrawn for illustration. Each coloured cell stands for a dense b x b block of token-to-token scores.

4.3 Published configurations

Parameter (base models, MLM)	BigBird-ITC	BigBird-ETC
Block length b	64	84
Global tokens g	2 x b = 128	256
Window length w	3 x b = 192	3 x b = 252
Random tokens r	3 x b = 192	0
Max. sequence length	4096	4096
Layers / heads / hidden size	12 / 12 / 768	12 / 12 / 768
Keys attended per query	**128 + 192 + 192 = 512**	**256 + 252 = 508**

Table 2. Hyperparameters from the paper's Table 8; the last row is derived by summing the components. Pre-training used 8 x 8 TPUv3 and 16 GB per chip with batch sizes of 32 to 64 reported for the memory-efficiency claim.

Notice that the ITC base model attends to about 512 keys per query, the same number of keys a BERT token sees in a 512-token input. It is applied to inputs of 4096 tokens, which is where the 8x figure comes from (4096 / 512 = 8).

5. Theoretical Guarantees

A distinctive feature of BigBird is that the sparsity pattern is justified, not just tested. The paper proves three results (Section 3 and appendices A to C of the paper).

Result	Statement (informal)	Why it matters
Universal approximation	A sparse-attention encoder can approximate any continuous sequence-to-sequence function on a compact domain, provided the attention graph contains a *star graph* (one global node connected to all).	Shows O(n) inner products can be as expressive as $O(n^2)$. Explains why global tokens are needed.
Turing completeness	Under the standard arbitrary-precision assumption, sparse encoder-decoder Transformers with O(n) inner products can simulate any Turing machine.	Sparse attention does not lose computational universality.
Limitation (no free lunch)	A task (finding the furthest vector for each vector) solvable by one full-attention layer needs roughly linear depth for any sparse pattern with near-linear edges, assuming the Orthogonal Vectors Conjecture.	Sparsity has a real cost on some tasks; it is not a drop-in replacement in all cases.

The limitation result applies to *any* sufficiently sparse mechanism, not only BigBird, and is a worst-case statement about a constructed task. It does not say that natural-language tasks suffer, but it is a reminder that sparse attention trades generality for scale.

6. Computational Complexity: BERT vs Longformer vs BigBird

	BERT (full attention)	Longformer	BigBird
Attention pattern	All pairs	Sliding window (optionally dilated) plus task-chosen global tokens	Sliding window + global tokens + random keys
Time / memory per layer	$O(n^2)$	O(n w)	O(n (g + w + r)), i.e. O(n) for fixed g, w, r
Typical max. input	512 tokens	4096 tokens in released models	4096 tokens in the paper's experiments
Long-range paths	Direct (1 hop)	Grows with depth: about w per layer	Short paths via global hubs and random edges
Theory	Universal approximator; Turing complete (prior work)	No formal expressiveness result in its paper	Universal approximator and Turing complete
Hardware friendliness	Dense, highly optimised	Needs custom banded kernel	Block-dense; random part needs gather

Table 3. Longformer entries are from Beltagy et al. (2020); the "no formal result" entry reflects that paper's focus on empirical results and is stated from my reading of it, not from a quote.

6.1 What this means numerically

At matched attention budgets the two sparse models cost about the same. If Longformer uses a window of 512 and BigBird-ITC attends to 512 keys per query (128 global, 192 window, 192 random), both perform about 512 score computations per token, which is 8x fewer than full attention at 4096 tokens and 32x fewer at 16,384 tokens. Their difference is therefore *which* 512 keys are chosen, not how many.

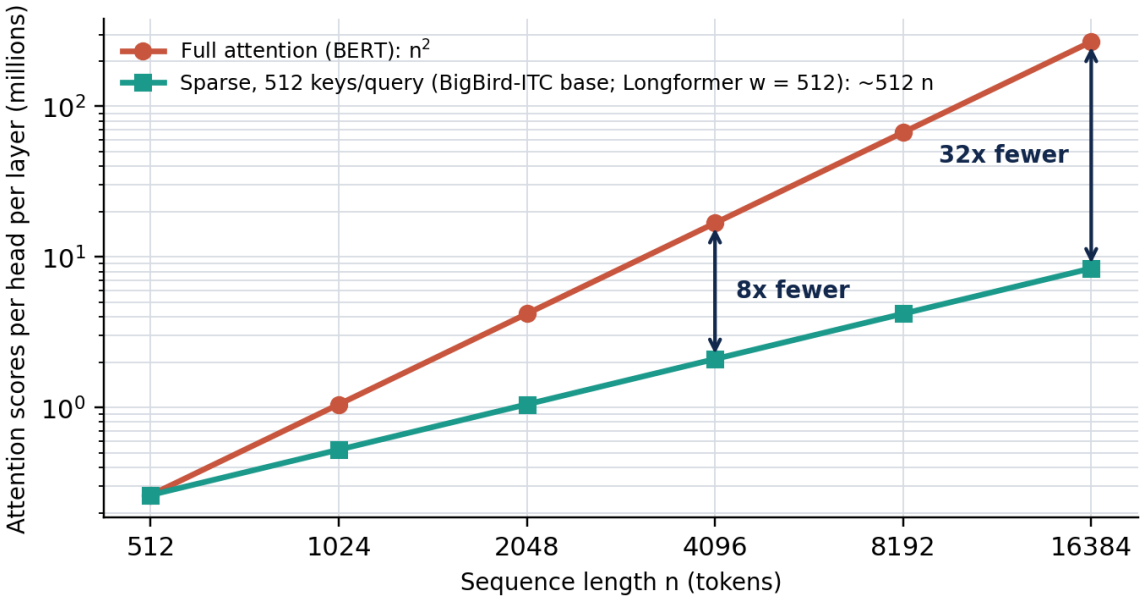


Figure 3. Illustrative scaling of attention scores per head per layer. Derived by arithmetic from n^2 versus 512 n; actual runtime also depends on kernels, batch size and hardware.

Why choose BigBird's pattern over a pure window? A window alone can only move information a distance of about w per layer, so distant tokens need many layers to interact. Global tokens give any two tokens a two-hop route (token to global to token), and random edges add expander-style shortcuts. The paper's results are consistent with this: BigBird-ETC beats Longformer on every QA dev metric in Table 4 below where Longformer is reported, and the ITC variant, which uses random attention, lands close to ETC (ahead on some metrics, behind on others).

7. Performance Characteristics

All numbers below are as reported by the BigBird paper. Base-size models are compared on dev sets for ablation-style comparisons; the large ETC model is compared on test leaderboards. Results are from 2020 and do not reflect later long-context models.

7.1 Question answering

Model (base, dev)	HotpotQA Ans	HotpotQA Sup	HotpotQA Joint	NQ LA	NQ SA	TriviaQA	WikiHop
RoBERTa (512)	73.5	83.4	63.5	-	-	74.3	72.4
Longformer	74.3	84.4	64.4	-	-	75.2	75.0
BigBird-ITC	**75.7**	86.8	67.7	70.8	53.3	**79.5**	75.9
BigBird-ETC	75.5	**87.1**	**67.8**	73.9	54.9	78.7	75.9

Table 4. QA dev results, base models (paper Table 2). F1 for HotpotQA, NQ and TriviaQA; accuracy for WikiHop. NQ = Natural Questions (LA = long answer, SA = short answer).

Test set (large)	Longformer	BigBird-ETC
HotpotQA Ans / Sup / Joint (F1)	81.2 / 88.3 / 73.2	81.2 / 89.1 / 73.6
Natural Questions LA / SA (F1)	-	77.8 / 57.9
TriviaQA Full / Verified (F1)	77.3 / 85.3	84.5 / 92.4
WikiHop (accuracy)	81.9	82.3

Table 5. Selected test-set results (paper Table 3). The paper reports BigBird-ETC as state of the art for NQ long answer, TriviaQA and WikiHop at submission time; on HotpotQA it ranked third by F1.

7.2 Long-document summarization

BigBird was used as a sparse *encoder* with a standard full-attention decoder. The authors justify this because summaries are short (median about 200 tokens) while inputs are long (median above 3000 tokens), so the encoder is the quadratic bottleneck.

Large models (ROUGE-1 / 2 / L)	arXiv	PubMed	BigPatent
Pegasus (re-evaluated)	43.85 / 16.83 / 39.17	44.53 / 19.30 / 40.70	52.25 / 33.04 / 41.80
BigBird-Pegasus	**46.63 / 19.02 / 41.77**	**46.32 / 20.65 / 42.33**	**60.64 / 42.46 / 50.01**

Table 6. Summarization on long documents (paper Table 4). The largest gain is on BigPatent, where salient content is spread across the whole document by design.

7.3 Language modelling, classification and genomics

Masked-LM bits per character (lower is better)	Base	Large
RoBERTa (length 512)	1.846	1.496
Longformer (length 4096)	1.705	1.358
BigBird-ITC (length 4096)	1.678	1.456
BigBird-ETC (length 4096)	**1.611**	**1.274**

Table 7. MLM held-out performance (paper Table 10). At large size, ITC is behind Longformer, while ETC is best.

- **Document classification:** gains are largest for long documents with few training examples. On arXiv the paper reports roughly a 5 percentage-point improvement over prior state of the art, while no gain appears on the short IMDb dataset.
- **Genomics:** with a 32K byte-pair vocabulary (about 8.78 base pairs per token) and 4096-token inputs, BigBird reduced DNA masked-LM bits per character from 1.23 (BERT, length 512) to 1.12, reached 99.9 F1 on promoter-region prediction (previous best 95.6), and improved histone-mark AUC on chromatin-profile prediction from 85.6 to 88.7. On DNase-hypersensitivity AUC it was marginally lower (92.1 vs 92.3).

8. Suitable Applications for Long Documents

BigBird is a good fit when the signal is distributed across thousands of tokens and the task can use an encoder over the whole input. The first four rows below were evaluated in the paper; the rest are natural extensions that follow from the same properties but were not benchmarked there.

Application	Why long context helps	Evidence
Multi-document / multi-hop QA	Evidence is spread across several passages; more retrieved text can be read at once	Paper (HotpotQA, WikiHop, NQ, TriviaQA)
Abstractive summarization of long texts	Salient content is distributed; input median above 3000 tokens	Paper (arXiv, PubMed, BigPatent)
Long-document classification	Discriminating cues may lie beyond the first 512 tokens	Paper (arXiv, Patents, etc.)
Genomics (promoters, chromatin profiles)	Functional effects in DNA are often non-local	Paper
Legal contracts, clinical notes, technical reports	Clauses or findings that refer to distant sections	Reasoned extension, not benchmarked

Choosing between variants. The paper's findings suggest BigBird-ETC (with structured global tokens) when you can define sensible global tokens, such as question tokens or paragraph markers, and BigBird-ITC when you need a general-purpose model with no task-specific design.

9. Advantages and Limitations

Advantages

- Linear scaling of attention cost with sequence length for fixed g, w, r.
- Up to 8x longer inputs on similar hardware, as reported by the authors.
- Retains theoretical expressiveness: universal approximation and Turing completeness.
- Strong empirical results on QA, long-document summarization and classification, and genomics.
- Block structure maps onto dense accelerator operations, so the sparsity yields real memory savings.
- Drop-in for the encoder of BERT-style and Pegasus-style models; warm-starting from RoBERTa was used.

Limitations

- Provable gap: some tasks need about linear depth under any sparse pattern (no free lunch).
- Gains shrink or vanish for short inputs; Hugging Face documentation advises full attention for short sequences.
- Random attention needs gather operations, which are less efficient than the dense parts.
- The sparse pattern is applied on the encoder side in the paper; decoders stay full-attention, which is fine for short outputs only.
- Implementation constraints: sequence length must be divisible by block size, and the Hugging Face code uses a fixed window of 3 blocks and 2 global blocks.
- ETC variants rely on hand-designed global tokens; ITC at large size lagged Longformer on MLM in the paper.

Context for today. BigBird was published in 2020. Since then, faster exact-attention kernels and other long-context techniques have changed the trade-offs, and sparse patterns like BigBird's are one option among several. I did not survey those newer developments for this review, so the comparisons here are limited to BERT, Longformer and BigBird as described in their original papers.

10. Conclusion

BigBird shows that full attention is not required to get the benefits of a Transformer on long inputs. By combining global tokens, local windows and random links, and by computing them in blocks, it turns quadratic cost into linear cost while keeping provable expressiveness. Compared with BERT it handles inputs eight times longer in the paper's setup; compared with Longformer it costs about the same at matched budgets but adds random connections and a theoretical foundation, and it performed better on most reported QA and summarization benchmarks. The trade-offs are real: sparse attention can need more depth on some tasks, offers little benefit on short texts, and depends on careful choice of global tokens.

References

[1] Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., Ahmed, A. (2020). Big Bird: Transformers for Longer Sequences. *NeurIPS 33*. arXiv:2007.14062.

[2] Beltagy, I., Peters, M. E., Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150.

[3] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.

[4] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 30*.

[5] Ainslie, J., et al. (2020). ETC: Encoding Long and Structured Data in Transformers. arXiv:2004.08483.

[6] Child, R., Gray, S., Radford, A., Sutskever, I. (2019). Generating Long Sequences with Sparse Transformers. arXiv:1904.10509.

[7] Kitaev, N., Kaiser, L., Levskaya, A. (2020). Reformer: The Efficient Transformer. *ICLR*.

[8] Yun, C., et al. (2020). Are Transformers Universal Approximators of Sequence-to-Sequence Functions? *ICLR*.

[9] Perez, J., Marinkovic, J., Barcelo, P. (2019). On the Turing Completeness of Modern Neural Network Architectures. arXiv:1901.03429.

[10] Watts, D. J., Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature* 393.

[11] Hugging Face Transformers documentation: BigBird model reference (attention_type, block_size, usage notes).